

Betaflight 커스텀 펌웨어

커스텀 빌드 설명서 (2025.12.5 v10)

CLI 파라미터 4종 + 코드 고정 기능(진입 래치 · 해제 대기 · 착륙 보조 · 호버 검증) + 보드별 slim 빌드

항목	내용
기준 소스	betaflight/betaflight tags/2025.12.5
관련 이슈	GitHub betaflight/betaflight #15775 (본인 제출)
저장소 / 브랜치	jsungho/betaflight2025.12.5_custom / custom-patch/alt-hold-throttle-range
대상 MCU	STM32F405 / STM32F7X2 / STM32H743 (보드별 slim 빌드)
대상 기체	MARIO5, AOS_UL7_O4, Mark4_6in, TJRC_10, 8IN-KOPI5_X8, CHIMERA7, AOS_UL7_X8, Explorer LR4, Pavo25 V2, X8_5INCH (총 10대, 3종 MCU)
현재 빌드	custom_v10_slim (hex 10개, custom-patch/firmware/2025.12.5/)
작성일 / 갱신	최초 2026-09-29 (v1~v3) / 최종 갱신 2026-10-02 (v10 + 빌드·검토·운용 부록)

이 문서는 커스텀 빌드에 추가된 CLI 파라미터 4종의 사용법, 관련 기준(공식) CLI 파라미터, 그리고 이후 단계(v4~v10)에서 코드에 고정한 기능(진입 스틱 래치, 해제 대기, 착륙 보조, 호버 스로틀 검증)과 보드별 빌드 구성·검토 결과·운용 메모를 정리한 기술 설명서입니다. 1~7장은 CLI 파라미터와 기준 설명, 8~13장은 버전별 부록, 14~16장은 빌드 구성, 코드 검토, 운용 메모입니다. 커스텀 빌드는 Betaflight 공식 지원 대상이 아니며, 사용자 자체 책임 하에 시험·적용합니다. 값이 충돌하면 번호가 큰(최신) 값이 우선합니다.

목차

1. 개요 및 배경 (버전 이력)
2. 신규 커스텀 CLI 파라미터 (2.1~2.4)
3. 관련 기존(공식) CLI 파라미터 (3.1~3.6)
4. 계산 공식 요약
5. 권장 설정 및 CLI 예시
6. 안전 점검 요약
7. 기체별 빌드/펌웨어 매핑
8. 부록: 스로틀 스틱 진입 래치 (v5)
9. 부록: Alt Hold 해제 대기 + OSD "ALT WAIT" (v6, v7)
10. 부록: 해제 대기 호버 구간 건너뛸 수정 (v7)
11. 부록: Alt Hold 착륙 보조 (v8)
12. 부록: alt_hold_hover_throttle 검증과 적용 범위 (v9)
13. 부록: 호버 변수 분리와 우선순위 정리 (v10)
14. 부록: 빌드 구성 · 제거 기능 · 배포 절차 (v4 포함)
15. 부록: 코드 검토 결과 · 착륙 보조 한계 · 컴파일 검증
16. 부록: 운용 메모 (랜딩 디스암, min_check, 실비행 확인) 및 버전 이력

1. 개요 및 배경

Alt Hold(고도유지)/Position Hold(위치유지) 비행 중 스로틀 스틱으로 하강을 조작할 때, 하강 방향의 유효 스틱 구간이 상승 방향보다 지나치게 좁아 정밀한 하강 제어가 어렵다는 문제를 GitHub 이슈 #15775로 제출했습니다. 공식 반영까지 시간이 걸릴 수 있어, 우선 자체 패치 펌웨어를 빌드해 시험 적용하기로 했습니다. 이후 실비행 안전성 검토 과정에서 EZ Disarm(착지 충격 자동 디스암) 기능이 비행 중에도 airmode 상태와 무관하게 작동해 오작동 위험이 있다는 점이 추가로 확인되어, v3에서 이를 개선하는 토글 파라미터를 추가했습니다. v4 이후에는 기체별 불필요 기능을 빼는 보드별 slim 빌드와 Alt Hold 동작 개선(코드 고정)을 단계적으로 추가했습니다.

CLI 파라미터 4종(v1~v3)은 모두 opt-in(기본값 OFF 또는 기존값 상속) 방식이라 값을 바꾸지 않는 한 공식 Betaflight와 동일하게 동작합니다. 반면 v5~v8의 코드 고정 기능(진입 래치, 해제 대기, 착륙 보조)은 CLI로 끌 수 없고 항상 작동합니다.

버전	추가 내용	PG 버전 변경
v1	alt_hold_full_low_is_max_descend, alt_hold_deadband_low	PG_ALTHOLD_CONFIG 4→5
v2	alt_hold_hover_throttle (Alt Hold/Position Hold 전용 호버 스로틀)	PG_ALTHOLD_CONFIG 5→6
v3	landing_disarm_airmode_off_only (EZ Disarm의 airmode 연동 토글)	PG_PID_PROFILE 11→12
v4	보드별 slim 빌드: 서보·배터리-컨티뉴·아날로그 OSD(MAX7456) 전 기체 제거 (14장)	없음
v5	Alt Hold 진입 스틱 래치(Entry Stick Latch) (8장)	없음
v6	Alt Hold 해제 대기(Exit Hold) + OSD "ALT WAIT" (9장)	없음
v7	해제 대기 호버 구간 건너뛰 결함 수정 + 호스트 시뮬레이션 테스트 (10장)	없음
v8	Alt Hold 착륙 보조(Landing Assist) + OSD "ALTHOLD : LANDING" (11장)	없음
v9	alt_hold_hover_throttle 값 검증(0 또는 1100~1700) + 스위치 Alt Hold/Position Hold 전용 (12장)	없음
v10	호버 변수 분리(override/captured) + 우선순위 정리 (13장)	없음

2. 신규 커스텀 CLI 파라미터

2.1 alt_hold_full_low_is_max_descend

항목	값
범위	OFF / ON
기본값	OFF (v1 도입, 기존 스톱 동작 유지)
소속(PG)	PG_ALTHOLD_CONFIG

설명: 스로틀 스틱이 min_check(THROTTLE_LOW) 미만일 때의 동작을 결정합니다. 스톱 Betaflight는 이 구간에서 스틱 위치와 무관하게 항상 강제 호버(고도 유지)로 처리합니다.

OFF(기본): THROTTLE_LOW 구간에서 stickFactor를 계산하지 않고 기본값 0.0(강제 호버)으로 둬 — 스톱과 동일.

ON: THROTTLE_LOW 구간을 '최대 하강 커맨드'(stickFactor = -1.0, 고정값)로 해석 — 스틱을 최하단까지 내리면 즉시 최대 하강률로 하강 시작.

실제 하강 강도는 이 토글이 아니라 alt_hold_climb_rate가 결정합니다 (3절 참고) — ON으로 켜도 climb_rate를 낮춰두면 하강 속도 자체는 완만합니다.

2.2 alt_hold_deadband_low

항목	값
범위	0 ~ 70 (%)
기본값	20
소속(PG)	PG_ALTHOLD_CONFIG

설명: 하강(throttle)측 데드밴드 폭을 상승측과 별개로 설정합니다. 기존 공식 `alt_hold_deadband`는 상승/하강 양쪽에 대칭 적용되었는데, 이 값이 하강측만 독립적으로 제어하도록 분리합니다.

동작 원리: 호버 스로틀을 기준으로 하강측 임계값(`lowThreshold`)을 결정 — 값이 클수록 스틱을 더 많이 내려야 하강이 시작되고(고정 호버 구간이 넓어짐), 값이 작을수록 미세한 스틱 하강에도 바로 반응합니다. 0%는 데드밴드 자체가 사라져 스틱 노이즈에도 하강 반응이 시작되므로 비추천, 30% 이상은 비례제어 구간이 압축되어 오히려 거칠어질 수 있어 비추천 — 기본값(20%) 근처($\pm 5\%$) 권장.

2.3 alt_hold_hover_throttle

항목	값
범위	허용값: 0 또는 1100 ~ 1700 (CLI 입력 범위는 0~1700이지만 1100 미만/1700 초과 0이 아닌 값은 무시되어 0으로 간주, v9)
기본값	0 (= <code>ap_hover_throttle</code> 상속)
소속(PG)	PG_ALTHOLD_CONFIG

설명: Alt Hold/Position Hold 전용 호버 스로틀 기준값입니다. 기존에는 GPS Rescue와 Alt Hold/Position Hold가 `ap_hover_throttle` 하나를 공유했는데, 이 값을 통해 Alt Hold/Position Hold만 독립적인 기준값을 가질 수 있습니다.

0(기본값): `ap_hover_throttle`을 그대로 상속 — 동작 변화 없음, 기존과 100% 동일.

1100~1700: 스위치 Alt Hold/Position Hold가 GPS Rescue·페일세이프와 다른 자기만의 호버 기준값 사용. 이 값은 `alt_hold_deadband/alt_hold_deadband_low` 데드밴드 계산의 중심점으로도 함께 쓰입니다.

v9/v10 적용 범위: 이 값은 조종자가 스위치로 켜 Alt Hold/Position Hold에서만 쓰입니다. 페일세이프 착륙(페일세이프도 ALT_HOLD_MODE를 켭니다)과 GPS Rescue는 이 값을 쓰지 않고 `ap_hover_throttle` > 진입 순간 스틱 캡처값(`ap_hover_throttle=0`일 때) > 기본값 1275 순으로 씁니다. 페일세이프가 해제되면 다시 이 값으로 돌아옵니다. 2025.12.5에는 AUTOPILOT_MODE가 없어 그 조건은 해당 없습니다. 자세한 내용은 12, 13장 참고.

2.4 landing_disarm_airmode_off_only (v3 신규)

항목	값
범위	OFF / ON
기본값	OFF (공식과 100% 동일 동작)
소속(PG)	PG_PID_PROFILE (프로파일별 설정 가능)

설명: 기존 공식 파라미터 `landing_disarm_threshold`(EZ Disarm, 착지 충격 감지 자동 디스암)의 발동 조건에 `airmode` 상태를 추가합니다.

OFF(기본): `airmode` 상태와 무관하게 작동 — 공식 Betaflight와 동일.

ON: `airmode`가 켜져 있으면 EZ Disarm 자체가 호출되지 않음(완전 비활성). `airmode`를 꺼야만 정상 작동합니다.

```
// pid.c pidController() 내부 (v3 패치)
if (pidRuntime.useEzDisarm &&
    (!pidRuntime.landingDisarmAirmodeOffOnly || !isAirmodeEnabled())) {
    disarmOnImpact();
}
```

사용 목적: AUX 스위치로 airmode를 켜고 끄며 운용하는 조종자(정상시 비행=ON, 착륙 시=OFF)의 경우, 이 값을 ON으로 해두면 정상시 비행 중엔 EZ Disarm이 완전히 꺼져 오작동 디스암 위험이 없고, 착륙을 위해 airmode를 끄는 순간부터만 EZ Disarm이 작동합니다.

주의(확인된 부수효과): 파일세이프/GPS Rescue 발동 시, airmode 스위치가 배정된 AUX 채널이 rxfail 설정으로 airmode ON 범위 값에 강제 고정되는 경우(보유 기체 다수 해당) 이 구간에서도 EZ Disarm이 자동으로 비활성됩니다. 자동복귀 중 거친 기동으로 인한 오작동 디스암을 방지하는 효과가 있어 의도적으로 허용하는 경우에만 ON으로 사용하는 것을 권장합니다.

프로파일별 설정: landing_disarm_threshold와 landing_disarm_airmode_off_only는 모두 PID 프로파일 값이라 set은 그때 선택된 프로파일에만 적용됩니다. 비행에 쓰는 프로파일마다 각각 설정/확인해야 합니다(예: profile 0/1/2 각각 get landing_disarm). 또한 landing_disarm_threshold = 0이면 EZ Disarm 자체가 꺼져 이 토글은 의미가 없습니다.

3. 관련 기준(공식) CLI 파라미터

위 커스텀 파라미터들은 아래 기준 공식 Betaflight 파라미터와 함께 동작하거나 상호작용합니다. 커스텀 값을 조정하기 전에 아래 항목들의 현재 설정을 먼저 확인하는 것을 권장합니다.

3.1 alt_hold_deadband / alt_hold_climb_rate

파라미터	범위	기본값	설명
alt_hold_deadband	0~70 (%)	20	상승(고throttle)측 데드밴드 폭. 커스텀 alt_hold_deadband_low와 쌍을 이룸
alt_hold_climb_rate	0~200	50 (=5m/s)	최대 상승/하강 속도. 실질적인 안전도를 결정하는 핵심 변수 — 값×10 = cm/s

커스텀 파라미터의 ON/OFF 자체보다 이 climb_rate 값이 실제 위험도를 좌우합니다. 기본 데드밴드 구간 공식상 THROTTLE_LOW 경계 바로 위에서 이미 stickFactor ≈ -0.87 (최대 하강의 87%)에 도달하므로, '안전장치'로 느껴지는 강제호버 구간은 매우 좁습니다. climb_rate를 보수적으로(15~20, 초당 1.5~2m) 낮추는 것이 alt_hold_full_low_is_max_descend의 ON/OFF보다 더 실질적인 안전 대책입니다.

3.2 ap_hover_throttle

GPS Rescue, 파일세이프 착륙, Alt Hold 해제 대기(9장 기준값), 그리고 (alt_hold_hover_throttle이 0이거나 범위 밖일 때) 스위치 Alt Hold/Position Hold가 참조하는 기준 호버 스로틀 값입니다. 기체마다 실측 호버 스로틀에 맞춰 CLI에 개별 설정되어 있습니다(예: Mark4_6in 1320). 유효한 alt_hold_hover_throttle(1100~1700)을 쓰면 스위치 Alt Hold/Position Hold만 이 값과 다른 기준을 갖게 되니, 두 값을 다르게 가져갈 명확한 이유가 있을 때만 분리 사용을 권장합니다.

3.3 min_check / max_check

파라미터	역할	커스텀 패치와의 관계
min_check (공식 기본 1050)	이 값 미만이면 THROTTLE_LOW로 분류 (아밍 안전, alt_hold_full_low_is_max_descend 등의 경계). 공식 2025.12.5 기본값 1050(pg/rx.c)	alt_hold_full_low_is_max_descend가 처리하는 구간의 하단 경계. 값을 바꾸면 이 경계도 함께 바뀜
max_check (공식 기본 1900)	이 값 이상이면 THROTTLE_HIGH로 분류 (아밍 안전 판정용). 실제 스틱 값을 클리핑하지 않음	PWM_RANGE_MAX(2000, 고정 상수)와는 별개 — 수신기가 실제 2000까지 출력하면 max_check 설정과 무관하게 stickFactor=1.0 완전 상승 가능

3.4 landing_disarm_threshold (EZ Disarm)

항목	값
범위	0~250 (0=비활성)
기본값	0 (권장 활성화는 대략 100 전후, 코드 주석 기준)

항목	값
내부 스케일	landingDisarmThreshold = 설정값 × 10.0, acc.jerkMagnitude(g/s 단위)와 직접 비교

착지 시 가속도 변화율(저크)이 이 임계값을 넘으면 자동으로 disarm합니다. 발동 조건은 다음과 같습니다 (pid.c disarmOnImpact()):

```
if (wasThrottleRaised()           // 이륙 이력(래치)
    && ((getMaxRcDeflectionAbs() < 0.05f // 스틱 중앙 ±5%
        && mixerGetRcThrottle() < 0.05f) // 스로틀 5% 미만
        || FLIGHT_MODE(ALT_HOLD_MODE))) { // 또는 Alt Hold 모드 중
    // Alt Hold 중 착지고도 이하이면 민감도 1.5배
    if (jerkMagnitude * sensitivity > landingDisarmThreshold) {
        disarm(DISARM_REASON_LANDING);
    }
}
```

v3의 landing_disarm_airmode_off_only는 이 함수를 호출할지 말지만 결정하며, 함수 내부 로직은 전혀 수정하지 않습니다.

조건	동작
landing_disarm_threshold = 0	EZ Disarm 비활성(공식 기본값). 이 경우 airmode_off_only도 의미 없음. 권장 기본 CLI는 45(5.1절)
임계값 > 0, airmode_off_only = OFF(기본)	airmode 상태와 무관하게 작동(공식과 동일)
임계값 > 0, airmode_off_only = ON	airmode가 켜져 있으면 호출 자체가 일어나지 않고, 꺼져 있을 때만 판정
이륙 이후 + 스틱 중앙·스로틀 5% 미만(일반 모드)	스로틀을 올려 둔 채로는 디스암되지 않음
Alt Hold 중	스로틀이 0이 아니어도 판정(호버 스로틀 때문). 착지 고도 이하면 저크 1.5배 민감
페일세이프/GPS Rescue 중	airmode 채널이 rxfail로 ON 고정이면 ON 설정 시 비활성

3.5 mixer_type = EZLANDING 및 ez_landing_* 계열

파라미터	범위	기본값	설명
mixer_type	LEGACY/EZLANDING 등	-	EZLANDING이면 착지 판단 시 모터 최대 출력을 강제로 제한
ez_landing_threshold	0~200	25	이 값 이하 스틱 편차일 때 착지로 간주 시작
ez_landing_limit	0~75 (%)	15	착지로 판단됐을 때 모터 최대 출력 상한
ez_landing_speed	0~250	50	이 속도 이하일 때 착지로 간주(GPS 속도 기준)

(2026-09-29 시점 기록) 당시 보유 기체 전체 mixer_type = EZLANDING 아님으로 변경(사용자 조치). 이후 권장 기본 CLI(5절, README 3-1절)는 EZLANDING(ez_landing_limit 10, ez_landing_threshold 30)으로 확정되어 현재 기준은 5절을 따름. 아래는 당시 비활성 사유(주의사항으로 유지): Alt Hold/Position Hold는 평상시에도 RPY 스틱이 중앙에 가까워 '착지 중' 오판 위험이 있고, alt_hold_full_low_is_max_descend = ON으로 급하강 후 PID가 급제동을 요구하는 순간 GPS 속도가 0에 가까워지며 스틱도 중앙이라 EZLANDING이 착지로 오판 → 제동에 필요한 추력 상한을 오히려 낮춰버릴 위험이 있었기 때문입니다.

3.6 feature AIRMODE / aux BOXAIRMODE

airmode 상태는 isAirmodeEnabled() = featureIsEnabled(FEATURE_AIRMODE) || IS_RC_MODE_ACTIVE(BOXAIRMODE)로 결정됩니다. 보유 기체 전체는 CLI에서 feature -AIRMODE(피쳐 자체 OFF)로 설정되어 있고, AUX 스위치에 BOXAIRMODE(permanentId 28)가 배정되어 있어 isAirmodeEnabled()는 곧 AUX 스위치 상태 그 자체입니다. 이 덕분에 landing_disarm_airmode_off_only가 사용자의 실제 조작 습관과 정확히 맞물립니다.

페일세이프/GPS Rescue 시에는 해당 AUX 채널이 rxfail 설정값(대부분 airmode ON 범위 내)으로 강제 고정되어, 신호 두절 중에는 항상 airmode = ON으로 처리됩니다(자동복귀 중 충분한 제어권한 확보 목적, Betaflight 표준 설계).
landing_disarm_airmode_off_only=ON 사용 시 이 구간에서 EZ Disarm이 함께 비활성됨을 인지하고 있어야 합니다.

4. 계산 공식 요약

Alt Hold 스틱 반응 계산 (alt_hold_multirotor.c)

```
lowThreshold = hoverThrottle - deadbandLow * (hoverThrottle - 1000)
highThreshold = hoverThrottle + deadband * (2000 - hoverThrottle)

rcThrottle < lowThreshold → stickFactor = scaleRangef(rcThrottle, 1000, lowThreshold, -1, 0)
lowThreshold~highThreshold → stickFactor = 0 (진짜 데드밴드, 고도 고정)
rcThrottle > highThreshold → stickFactor = scaleRangef(rcThrottle, highThreshold, 2000, 0, 1)

targetVelocity = stickFactor * maxVelocity // cm/s, 기본 maxVelocity = climb_rate * 10
```

hoverThrottle(v10 기준): 유효한 alt_hold_hover_throttle(1100~1700)이 있고 failsafelsActive()도 GPS_RESCUE_MODE도 아닐 때는 그 값, 그 외에는 ap_hover_throttle > 진입 순간 스틱 캡처값(ap_hover_throttle=0일 때) > 기본값 1275 순입니다(13장). maxVelocity(v8 기준): Alt Hold + Airmode OFF + 시동 중이고 고도 5m 이하면 gps_rescue_descend_rate×2, 2m 이하면 ×1로 교체됩니다(11장). min_check 미만(THROTTLE_LOW) 구간은 이 공식이 아니라 alt_hold_full_low_is_max_descend 값으로 별도 결정됩니다(OFF=0 고정, ON=-1.0 고정).

EZ Disarm 판정 (pid.c)

```
landingDisarmThreshold = landing_disarm_threshold(CLI) * 10.0
jerkMagnitude = |가속도 변화율| (g/s 단위, sensors/acceleration.c)
sensitivity = (ALT_HOLD_MODE 중 && 착지고도 이하) ? 1.5 : 1.0

jerkMagnitude * sensitivity > landingDisarmThreshold → disarm(DISARM_REASON_LANDING)
```

5. 권장 설정 및 CLI 예시

5.1 권장 기본 CLI 값 (전 기체 공통, 2026-10-02 사용자 확정)

플래시·diff all 복원 후 아래 값을 기본값으로 적용합니다(사용자 확정). 프로파일 값(landing_disarm_*, ez_landing_*)은 프로파일마다 적용해야 합니다.

파라미터	값	설명
gps_ublox_flight_model	AIRBORNE_1G	u-blox GPS 동적 모델(저가속 항공기용)
mixer_type	EZLANDING	EZ Landing 모터 믹서(착륙 시 모터 출력 제한)
ez_landing_limit / ez_landing_threshold	10 / 30	EZ Landing 출력 제한 / 시작 임계값
min_check	1050	스로틀 로우 판정(공식 기본값과 동일)
alt_hold_climb_rate	70	Alt Hold 최대 속도 스케일(×10 cm/s)
alt_hold_deadband	25	상승 쪽 데드밴드
alt_hold_deadband_low	0	하강 쪽 데드밴드 없음(스로틀을 호버에서 내리면 곧바로 하강 시작)
alt_hold_full_low_is_max_descend	ON	스로틀 min_check 미만이면 최대 하강 속도로 하강(2.1절)
alt_hold_hover_throttle	1400	스위치 Alt Hold/Position Hold 전용 호버(유효값, 12~13장)

파라미터	값	설명
gps_rescue_descend_rate	135	GPS Rescue 하강 속도. 착륙 보조(11장)의 속도 기준으로도 재사용됨
gps_rescue_disarm_threshold	60	GPS Rescue 착지 후 디스암 임계값
gps_rescue_use_mag / pos_hold_without_mag	ON / OFF	GPS Rescue는 자력계 헤딩 사용 / Position Hold는 자력계 필수
landing_disarm_threshold	45	EZ Disarm(착지 충격 자동 디스암) 활성화. 값이 낮을수록 민감(코드 주석 안전값 약 100). 프로파일별 값
landing_disarm_airmode_off_only	ON	airmode가 켜져 있으면 EZ Disarm 호출 안 함(airmode OFF 착륙 때만 작동). 프로파일별 값

```

set gps_ublox_flight_model = AIRBORNE_1G
set mixer_type = EZLANDING
set ez_landing_limit = 10
set ez_landing_threshold = 30
set landing_disarm_threshold = 45
set gps_rescue_descend_rate = 135
set gps_rescue_disarm_threshold = 60
set alt_hold_deadband = 25
set alt_hold_climb_rate = 70
set min_check = 1050
set gps_rescue_use_mag = ON
set pos_hold_without_mag = OFF
set alt_hold_deadband_low = 0
set alt_hold_full_low_is_max_descend = ON
set alt_hold_hover_throttle = 1400
set landing_disarm_airmode_off_only = ON

```

이 값이 이 문서의 기준 권장값입니다. landing_disarm_threshold(45)와 airmode_off_only는 PID 프로파일 값이므로 비행에 쓰는 프로파일마다 적용하세요. 자력계가 없는 Pavo25 V2는 gps_rescue_use_mag/pos_hold_without_mag가 해당 없습니다. 이전 권장값(landing_disarm_threshold 0)은 폐기되었습니다.

Alt Hold 시험 설정 (보수적 시작값)

```

set alt_hold_climb_rate = 15          # 초당 1.5m로 보수적으로 시작
set alt_hold_deadband_low = 20        # 기본값 유지 권장
set alt_hold_full_low_is_max_descend = OFF # 1단계: 데드밴드 분리만 우선 검증
set alt_hold_hover_throttle = 0       # 처음엔 상속(0)으로 검증
save

```

EZ Disarm + airmode 연동 설정 예시 (권장 기본은 5.1절의 45, 아래는 민감도를 바꿔 볼 때의 예)

```

profile 1
set landing_disarm_threshold = 50      # 착지 충격 판정 임계값(민감도)
set landing_disarm_airmode_off_only = ON # airmode OFF일 때만 EZ Disarm 작동
profile 2
set landing_disarm_threshold = 50
set landing_disarm_airmode_off_only = ON
save

```

landing_disarm_threshold의 코드 주석은 "relatively safe values are around 100"이라고 안내합니다. 50은 상대적으로 민감한 값이므로, 처음엔 70~80 정도로 시작해 블랙박스로 실제 착지 충격 크기를 확인한 뒤 필요시 낮추는 것을 권장합니다.

6. 안전 점검 요약

점검 항목	결과
아밍 전 ALTHOLD/POSHOLD 스위치 ON 상태 → 아밍 차단	기존 표준 기능, 패치와 무관. 스위치 OFF로 아밍 후 이륙 후 ON 권장
wasThrottleRaised() 래치로 인한 모드 유지	문제 없음 — 이륙 후 스로틀이 낮아져도 모드가 임의로 꺼지지 않음
EZ Disarm과 Position Hold의 관계	다수 기체에서 ALTHOLD/POSHOLD가 같은 AUX 스위치에 배정되어 있어 Position Hold 활성화 시 Alt Hold도 함께 활성화됨 — 패치 로직이 Position Hold 중에도 동작
mixer_type = EZLANDING과의 상호작용	2026-09-29 시점에는 위험 확인으로 비활성화했으나, 이후 권장 기본값은 EZLANDING으로 확정(5장). 급하강 후 제동 구간의 오판 가능성은 주의 항목으로 유지
EZ Disarm ↔ airmode 상시 연동 위험	v3의 landing_disarm_airmode_off_only로 해결 — 프로파일별 설정이므로 쓰는 프로파일마다 ON 확인
페일세이프/GPS Rescue 중 EZ Disarm 상태	landing_disarm_airmode_off_only=ON 시 이 구간에서도 비활성(코드 검증 완료) — 자동복귀 중 오작동 방지 목적으로 사용자가 의도적으로 수용
해제 대기(v6/v7) 중 스로틀 하강 불가	설계된 동작(자세는 앵글 유지, 스로틀을 내려도 고도 유지). 비상 시에는 디스암 스위치 사용(9장, 16장)
착륙 보조(v8)의 한계	속도 상한은 목표고도 이동 속도 제한일 뿐 실제 하강속도의 직접 제한이 아님, 고도 기준은 시동 지점 (15장)
alt_hold_hover_throttle 오입력(v9/v10)	1100 미만/1700 초과 값은 무시되어 ap_hover_throttle 사용. 페일세이프 착륙/GPS Rescue에는 적용되지 않음(12, 13장)

7. 기체별 빌드/펌웨어 매핑 (v10)

v4부터 통합 타겟(MCU 단위) hex가 아니라 기체(보드)별 slim hex를 배포합니다. 파일명 형식: `betafight_2025.12.5_<MCU>_<BOARD>_<VEHICLE>_custom_v10_slim.hex`. 각 기체에 맞는 파일 하나만 올립니다.

기체	보드	MCU	자력계	Flash	비고
MARIO5	SPEEDYBEEF405V4	F405	포함	39.43%	CRSF
AOS_UL7_O4	SPEEDYBEEF405V4	F405	포함	39.21%	FPort + SmartPort
Mark4_6in	JHEF405PRO	F405	포함	39.74%	SBUS, 텔레메트리 없음
TJRC_10	MATEKF722SE	F722	포함	79.01%	듀얼자이로, LED 유지
8IN-KOPIS_X8	SPEEDYBEEF7V3	F722	포함	76.49%	OCTOX8 믹서
CHIMERA7	FLYWOOF722PROV2	F722	포함	74.21%	
AOS_UL7_X8	MATEKF722HD	F722	포함	76.79%	OCTOX8 믹서, LED 유지
Explorer LR4	JHEF7DUAL	F722	포함	76.85%	듀얼자이로, EZ Disarm 활성화 기체
Pavo25 V2	JHEF7DUAL	F722	없음	73.66%	센서 미장착, PINIO 사용
X8_5INCH	MATEKH743	H743	포함	24.38%	OCTOX8 믹서, FPort

같은 MCU라도 보드·기체별로 컴파일 옵션이 달라 hex가 서로 다릅니다(이전 통합 빌드의 "hex 동일" 설명은 폐기). 제조사별 보드 리소스 매핑(핀 배선)은 CLI 설정이 결정하는 Unified Target 구조이므로 플래시 후 반드시 각 기체 자신의 DIFF ALL을 재적용해야 합니다. F722 보드는 512KB 플래시라 공식 조건상 Alt Hold/Position Hold/GPS가 기본 제외되어, 빌드 스크립트가 `-DUSE_ALTITUDE_HOLD -DUSE_GPS -DUSE_POSITION_HOLD`로 다시 겁니다(14장).

본 문서는 커스텀 빌드 사용을 위한 참고 자료이며, 실비행 적용은 프롭 제거 벤치 테스트 → 개활지 저고도 단계적 비행 시험을 거쳐 사용자 자체 판단과 책임 하에 진행합니다.

8. 부록: 스로틀 스틱 진입 래치 (Entry Stick Latch) — 2025.12.5 v5

2025.12.5 커스텀 빌드(브랜치 custom-patch/alt-hold-throttle-range, 보드별 slim 빌드 v5)에 새로 추가된 기능입니다. CLI 파라미터는 아님(코드에 고정, PG 버전 변경 없음) — 이 문서의 2절 4종 CLI 파라미터와는 별개로 동작합니다.

8.1 배경 및 문제

Alt Hold/Position Hold에 진입하는 순간 조종자의 스로틀 스틱이 호버 위치에서 살짝 벗어나 있으면, 기존 로직은 그 스틱 위치를 곧바로 alt_hold_deadband/alt_hold_deadband_low 기준과 비교해 즉시 상승·하강 커맨드로 해석할 수 있습니다. 즉, 모드 진입 직후 스틱이 정확히 중앙(호버)에 있지 않으면 조종자가 의도하지 않은 고도 변화가 시작될 수 있습니다.

8.2 동작

Alt Hold 진입 순간(altHoldProcessTransitions에서 비활성→활성 전환 시점)의 rcCommand[THROTTLE] 값을 entryThrottle로 저장하고, entryLatched = true로 래치를 건다. 래치가 걸려 있는 동안은 스틱 위치와 무관하게 고도를 그대로 유지하며(stickFactor = 0), alt_hold_full_low_is_max_descend도 이 구간에서는 적용되지 않는다. 스틱이 진입 시점 값에서 PWM 기준 5% (1000~2000 범위의 5% = 50, 예: 진입 시 1500이면 1450 이하 또는 1550 이상) 이상 움직이면 래치가 풀리고, 그 이후부터는 기존 커스텀 로직(alt_hold_hover_throttle, alt_hold_deadband, alt_hold_deadband_low, alt_hold_full_low_is_max_descend)이 호버 스로틀 대비 스틱의 '절대 위치' 기준으로 그대로 적용된다. 한 번 풀린 래치는 Alt Hold를 꺾다가 다시 켜기 전까지 재적용되지 않는다. 페일세이프/GPS Rescue 하강 오버라이드는 래치 상태와 무관하게 그대로 최우선 적용된다(failsafelsActive() 분기가 래치 로직 이후에 무조건 stickFactor를 덮어씀).

8.3 구현 (src/main/flight/alt_hold_multirotor.c)

```
#define ALT_HOLD_ENTRY_LATCH_RELEASE_PWM  (0.05f * (PWM_RANGE_MAX - PWM_RANGE_MIN))

// altHoldProcessTransitions(): 비활성→활성 전환 시
altHold.entryThrottle = rcCommand[THROTTLE];
altHold.entryLatched = true;

// altHoldUpdateTargetAltitude(): 매 틱
if (altHold.entryLatched &&
    fabsf(rcCommand[THROTTLE] - altHold.entryThrottle) >= ALT_HOLD_ENTRY_LATCH_RELEASE_PWM) {
    altHold.entryLatched = false;
}
if (altHold.allowStickAdjustment && !altHold.entryLatched) {
    // 기존 stickFactor 계산 (데드밴드/최대하강 포함)
}
```

8.4 원본 출처 및 이식

jsungho/betaflight2026.6.x_custom 저장소 브랜치 custom-patch/alt-hold-throttle-range-2026.6.2, 커밋 665f62a9e ("Alt Hold: hold altitude on entry until throttle stick moves 5%")의 src/main/flight/alt_hold_multirotor.c 변경을 이식했습니다. 2025.12.5는 alt_hold.h/alt_hold_multirotor.c 코드 구조(변수명·함수명)가 2026.6.2와 달라(예: autopilotCaptureHoverThrottleForAltHold() 등 2026.6.2 전용 함수 부재) 동일한 altHoldState_t 구조체에 entryLatched/entryThrottle 필드를 추가하고, 이 저장소의 altHoldProcessTransitions / altHoldUpdateTargetAltitude 함수에 같은 동작을 재현하는 방식으로 이식했습니다.

8.5 영향받는 파라미터

alt_hold_hover_throttle, alt_hold_deadband, alt_hold_deadband_low, alt_hold_full_low_is_max_descend(2절 참고) — 이 값들은 래치가 풀린 이후부터 적용되며, 값 자체나 기본값은 변경되지 않았습니다. Position Hold의 좌우/전후 스틱 로직은 이번 패치의 영향을 받지 않습니다.

8.6 검증

전 기체(10대) 보드별 slim 재빌드 성공, 컴파일 경고/오류 없음, 플래시 오버플로 없음. 4종 커스텀 CLI 파라미터 문자열(alt_hold_deadband_low / alt_hold_full_low_is_max_descend / alt_hold_hover_throttle / landing_disarm_airspeed_off_only) 및 자력계 관련 문자열이 hex 바이너리에 그대로 포함됨을 확인. #pragma message 진단 삽입(임시, 검증 후 원복)으로 ALT_HOLD_ENTRY_LATCH_RELEASE_PWM이 실제로 전처리 단계에서 정의됨을 확인. 파일명 접미사는 custom_v4_slim → custom_v5_slim으로 갱신.

8.7 권장 시험 절차

① 프롭 제거 벤치에서 Alt Hold 진입 시 스로틀 스틱을 진입 위치 근처에서 작게($\pm 5\%$ 미만) 흔들어도 고도 목표가 변하지 않는지(로그상 targetAltitudeCm 고정) 확인. ② 스틱을 5% 이상 움직였을 때만 상승/하강 커맨드가 반응하기 시작하는지 확인. ③ 한 번 래치가 풀린 뒤 스틱을 다시 진입 위치로 돌려도 래치가 재적용되지 않는지(모드를 껐다 켜야만 재래치) 확인. ④ 블랙박스 rcCommand[THROTTLE]과 목표고도 궤적을 함께 로깅해 래치 해제 시점을 사후 검증.

9. 부록: Alt Hold 해제 대기(Exit Hold) + OSD "ALT WAIT" 표시 — 2025.12.5 v6

2025.12.5 커스텀 빌드(브랜치 custom-patch/alt-hold-throttle-range, 보드별 slim 빌드 v6)에 새로 추가된 기능입니다. CLI 파라미터는 아님(코드에 고정, PG 버전 변경 없음). 8장의 진입 스틱 래치(v5)와는 별개 기능이며, 둘은 함께 동작합니다.

9.1 배경

기존에는 Alt Hold 스위치를 끄는 순간 즉시 모드가 해제되어, 그 순간 스로틀 스틱이 호버 위치와 다르면 추력이 갑자기 바뀔 수 있었습니다.

9.2 동작

Alt Hold 스위치를 끈 순간에 Alt Hold가 활성 상태였다면 즉시 해제하지 않고 '해제 대기' 상태로 전환해 고도를 계속 유지합니다. 대기 중에는 스틱에 의한 고도 조절을 완전히 무시합니다. 스로틀 스틱이 $ap_hover_throttle \pm 5\%$ (PWM 50, 1000~2000 범위) 안에 들어오면 그 순간 Alt Hold를 완전히 해제합니다. 기준은 $ap_hover_throttle$ 만 사용하며, $alt_hold_hover_throttle$ 이나 thr_mid 는 기준이 아닙니다. $ap_hover_throttle$ 이 0이면 Alt Hold의 실호 호버 값으로 대체합니다($alt_hold_hover_throttle$ 이 설정돼 있으면 그 값, 아니면 $ap_hover_throttle$ 상속분과 동일). 스위치를 끈 순간 스틱이 이미 그 구간 안에 있으면 대기 없이 바로 해제됩니다. 대기 중 스위치를 다시 켜면 대기를 취소하고 일반 Alt Hold로 복귀하며, 이때 진입 스틱 래치(8장)가 현재 스틱 위치로 새로 걸립니다. 파일세이프/GPS Rescue 하강 오버라이드는 대기 상태와 무관하게 항상 최우선 적용됩니다.

9.3 OSD 표시

OSD 비행모드 요소(osdElementFlymode)에서 해제 대기 중이면 "ALT WAIT"를 경고색(DISPLAYPORT_SEVERITY_WARNING)으로 표시합니다. 표시 우선순위: !FS!(파일세이프) → RESC(GPS Rescue) → HEAD(헤드프리) → PASS(패스스루) → ALT WAIT → POSH(Position Hold) → ALTH(Alt Hold).

9.4 구현

```
// alt_hold_multirotor.c
#define ALT_HOLD_EXIT_HOVER_BAND_PWM (0.05f * (PWM_RANGE_MAX - PWM_RANGE_MIN))

bool altHoldRequestActive(bool switch0n)
{
    const bool wasSwitch0n = altHold.prevSwitch0n;
    altHold.prevSwitch0n = switch0n;
    if (switch0n) {
        if (altHold.exitPending) {
            altHold.exitPending = false;
            altHold.entryThrottle = rcCommand[THROTTLE];
            altHold.entryLatched = true;
        }
        return true;
    }
    if (wasSwitch0n && altHold.isActive) {
        altHold.exitPending = true;
    }
    if (altHold.exitPending) {
        const uint16_t apHover = autopilotConfig()->hoverThrottle;
        const float hoverPwm = apHover != 0 ? (float)apHover : altHold.hoverThrottle;
        if (fabsf(rcCommand[THROTTLE] - hoverPwm) <= ALT_HOLD_EXIT_HOVER_BAND_PWM) {
            altHold.exitPending = false;
        }
    }
}
return altHold.exitPending;
```

```

}

// fc/core.c: IS_RC_MODE_ACTIVE(BOXALTHOLD) -> ALT_HOLD_SWITCH_REQUEST()
// (!USE_WING: altHoldRequestActive(...), USE_WING: 기존 그대로)
// DISABLE_FLIGHT_MODE(ALT_HOLD_MODE) 분기에서 altHoldClearExitPending() 호출

// osd/osd_elements.c: osdElementFlymode()에 ALT WAIT 분기 추가
// (PASS 다음, POSH 이전)

```

9.5 원본 출처 및 이식

jsungho/betaflight2026.6.x_custom 저장소 브랜치 custom-patch/alt-hold-throttle-range-2026.6.2, 커밋 b83985b("Alt Hold: keep holding after switch-off until throttle reaches hover +/-5%, OSD ALT WAIT") 및 커밋 e4593eb("Alt Hold exit hold: use ap_hover_throttle as reference")의 src/main/flight/alt_hold_multitrotor.{c,h}, src/main/fc/core.c, src/main/osd/osd_elements.c 변경을 이식했습니다. 진입 스틱 래치 커밋(665f62a, 8장)은 이번 이식 대상이 아닙니다. 2026.6.2는 autopilotGetEffectiveHoverThrottlePwm()(2026.6.2 전용 함수)을 폴백으로 썼지만 2025.12.5에는 이 함수가 없어, 이 저장소가 v2 패치에서 이미 계산해 쓰는 altHold.hoverThrottle(alt_hold_hover_throttle 이 설정돼 있으면 그 값, 아니면 ap_hover_throttle 상속)을 동일한 역할의 폴백으로 사용했습니다. fc/core.c는 2026.6.2의 AUTOPILOT_MODE/flightPlanNavIsRescueDescentActive() 조건이 2025.12.5에는 없어(해당 기능 자체 없음) 제외하고, IS_RC_MODE_ACTIVE(BOXALTHOLD) -> ALT_HOLD_SWITCH_REQUEST() 치환만 반영했습니다.

9.6 검증

전 기체(10대) 보드별 slim 재빌드 성공, 컴파일 경고/오류 없음, 플래시 오버플로 없음(24.35~78.90% 사용). 4중 커스텀 CLI 파라미터, 자력계 관련 문자열, "ALT WAIT" 문자열이 전 기체 hex 바이너리에 포함됨을 확인. #pragma message 진단 삽입(임시, 검증 후 원복)으로 ALT_HOLD_EXIT_HOVER_BAND_PWM이 실제로 전처리기 단계에서 정의됨을 확인. 파일명 접미사는 custom_v5_slim -> custom_v6_slim으로 갱신.

9.7 권장 시험 절차

① 프롭 제거 벤치에서 Alt Hold 중 스위치를 끄되 스틱이 호버 위치에서 벗어나 있는 상태로 시험 — 스위치를 꺼도 고도가 유지되고(targetAltitudeCm 고정), OSD에 ALT WAIT가 뜨는지 확인. ② 스틱을 ap_hover_throttle $\pm 5\%$ 안으로 가져왔을 때 그 순간 해제되고 OSD 표시가 사라지는지 확인. ③ 대기 중 스위치를 다시 켰을 때 정상적으로 Alt Hold로 복귀하고(진입 스틱 래치가 새로 걸림) 대기 상태가 취소되는지 확인. ④ 스위치를 끄는 순간 스틱이 이미 호버 근처였을 때 대기 없이 즉시 해제되는지 확인. ⑤ 블랙박스로 rcCommand[THROTTLE], ap_hover_throttle, 목표고도, ALT WAIT 표시 구간을 함께 로깅해 해제 타이밍을 사후 검증.

10. 부록: 해제 대기 — 호버 구간 건너뛸 결함 수정 + 시뮬레이션 테스트 — 2025.12.5 v7

9장(해제 대기, v6)에 이어지는 수정이다. CLI 파라미터는 아님(코드에 고정, PG 버전 변경 없음).

10.1 문제

v6의 해제 대기는 매 사이클(100Hz, 10ms 간격) 스로틀 스틱이 $ap_hover_throttle \pm 5\%$ (PWM 50) 구간 안에 '머무르는지'만 검사했다. 조종자가 스틱을 매우 빠르게 움직이면 한 사이클과 다음 사이클 사이에 스틱 값이 이 구간을 통째로 건너뛸 수 있다 — 예를 들어 한 틱에는 구간보다 한참 위(예: 1600), 다음 틱에는 구간보다 한참 아래(예: 1000)로 가는 경우, 구간 안에 정확히 머무른 샘플이 한 번도 없어 Alt Hold가 영원히 해제되지 않는 결함이 있었다.

10.2 수정

해제 대기가 시작되는 순간과 매 사이클마다 직전 스로틀 값(exitPrevThrottle)을 저장해두고, (스틱 - ap_hover_throttle)의 부호가 직전 샘플과 이번 샘플 사이에서 바뀌었으면(=호버 값을 가로질렀으면) 구간 안에 정확히 들어오지 않았어도 그 자리에서 해제한다. 위쪽에서 아래로, 아래쪽에서 위로 넘어가는 경우 모두 해당한다. 해제 조건 = $|스틱 - ap_hover_throttle| \leq 50$ 이거나, 직전 사이클 대비 호버 값을 가로지른 경우.

```
// alt_hold_multirotor.c
typedef struct {
    ...
    float exitPrevThrottle; // v7: 직전 스로틀 샘플(호버 가로지름 판정용)
} altHoldState_t;

bool altHoldRequestActive(bool switchOn)
{
    ...
    if (wasSwitchOn && altHold.isActive) {
        altHold.exitPending = true;
        altHold.exitPrevThrottle = rcCommand[THROTTLE];
    }
    if (altHold.exitPending) {
        const uint16_t apHover = autopilotConfig()->hoverThrottle;
        const float hoverPwm = apHover != 0 ? (float)apHover : altHold.hoverThrottle;
        const float prevDelta = altHold.exitPrevThrottle - hoverPwm;
        const float delta = rcCommand[THROTTLE] - hoverPwm;
        altHold.exitPrevThrottle = rcCommand[THROTTLE];
        if (fabsf(delta) <= ALT_HOLD_EXIT_HOVER_BAND_PWM
            || (prevDelta < 0.0f) != (delta < 0.0f)) {
            altHold.exitPending = false;
        }
    }
    return altHold.exitPending;
}
```

10.3 해제 대기 중 자세/스로틀 동작 (설계 확인)

해제 대기 중에도 ALT_HOLD_MODE 자체는 계속 켜진 상태다. 즉 자세 제어는 앵글(자동 수평) 모드 그대로 유지되고, 대기 중 스로틀 스틱을 아무리 내려도 고도가 내려가지 않는다(목표 고도를 그대로 유지). 이는 설계된 동작이다 — 스틱이 호버 구간에 들어오거나 호버 값을 가로지르기 전까지는 스틱 입력을 의도적으로 완전히 무시한다. 처음 접하면 '스로틀을 내렸는데 안 떨어진다'고 당황할 수 있으므로 비행 전 반드시 숙지한다.

10.4 시뮬레이션 테스트

jsungho/betaflight2026.6.x_custom 저장소 브랜치 custom-patch/alt-hold-throttle-range-2026.6.2, 커밋 415e8c4("Alt Hold exit hold: release when stick crosses hover (fast flick); sim tests")의 AltholdCustomSim 테스트 스위트(해제 대기 관련 부분)를 src/test/unit/althold_unittest.cc에 이식했다. 2025.12.5는 2026.6.2 전용

API(`autopilotGetEffectiveHoverThrottlePwm` 등)가 없어 이 저장소의 실제 `pg/autopilot.h`, `pg/alt_hold.h` 구조(`autopilotConfigMutable`, `altHoldConfigMutable`, `rxConfigMutable()`→`mincheck` 등)에 맞춰 스텝을 다시 구성했다. 호스트(PC, x86) 빌드라 실제 타깃 빌드와 무관하게 `cd src/test && make test_althold_unittest` 로 바로 실행할 수 있다.

검증한 13개 시나리오(전부 통과): 진입 스틱 래치가 고도를 유지/해제하는 경우(3개, v5), 해제 대기가 호버 구간 밖에서는 고도를 유지하다가 `ap_hover_throttle ±5%` 진입 시 해제, 기준이 `alt_hold_hover_throttle`이 아니라 `ap_hover_throttle`임을 구분, 스위치를 끈 순간 스틱이 이미 구간 안이면 즉시 해제, 빠른 스틱 이동으로 구간을 위→아래/아래→위로 건너뛰어도 해제(v7 수정 대상), 대기 중 스위치 재투입 시 재래치, 디스암 시 대기 삭제, Alt Hold에 진입한 적이 없는 상태에서 스위치를 끄면 대기가 시작되지 않는 경우.

10.5 빌드/바이너리 검증

전 기체(10대) 보드별 slim 재빌드 성공(파일명 접미사 `custom_v6_slim` → `custom_v7_slim`), 컴파일 경고/오류 없음, 플래시 오버플로 없음(24.35~78.92% 사용, v6 대비 사실상 동일 — float 필드 1개와 분기 하나 추가라 증가분 미미). 4종 커스텀 CLI 파라미터와 "ALT WAIT" 문자열이 전 기체 hex 바이너리에 포함됨을 확인. `#pragma message` 진단 삽입(임시, 검증 후 원복)으로 `ALT_HOLD_EXIT_HOVER_BAND_PWM`이 v7 코드에서도 전처리기 단계에서 실제로 정의됨을 확인.

10.6 권장 시험 절차 (9.7절에 추가)

① 프롭 제거 벤치에서 Alt Hold 해제 대기 중 스틱을 천천히 호버 근처로 가져와 정상 해제되는지 확인(9.7절 기존 절차). ② 해제 대기 중 스틱을 한쪽 끝에서 반대쪽 끝으로 매우 빠르게(플릭) 움직여, 호버 구간에 머무는 시간 없이도 그 순간 해제되는지 위/아래 양방향 모두 확인 — 이것이 v7에서 새로 고친 부분이다. ③ 해제 대기 중 스로틀을 완전히 내려도 고도가 유지되고 하강하지 않는지(설계대로) 확인. ④ 블랙박스 `rcCommand[THROTTLE]`, `ap_hover_throttle`, 목표고도, ALT WAIT 표시 구간을 함께 로깅해 실제 가로지름 판정 타이밍을 사후 검증.

11. 부록: Alt Hold 착륙 보조(Landing Assist) — 2025.12.5 v8

10장(해제 대기, v7)에 이어지는 추가 기능이다. CLI 파라미터는 아님(코드에 고정, PG 버전 변경 없음). 원본: jsungho/betaflight2026.6.x_custom 저장소 브랜치 custom-patch/alt-hold-throttle-range-2026.6.2, 커밋 90d5d9c(착륙 보조 최초), f29a459(OSD 고도 조건), 0f78e2e(5.0m/5.5m 히스테리시스), e2c4e83(1.8m/2.2m 히스테리시스)를 2025.12.5 코드 구조에 맞게 이식했다(최종 상태 기준으로 이식).

11.1 배경

Alt Hold 비행 중 조종자가 Airmode를 끄고 착륙 조작을 하는 운용(2장 권장 기본값의 전제)에서, 지면 근처에서도 alt_hold_climb_rate로 설정한 속도 그대로 상승·하강할 수 있어 과도한 접지 충격이나 튀어오름이 발생할 위험이 있었다. 착륙 보조는 이 구간에서만 수직 속도 상한을 GPS Rescue의 하강 속도 설정(gps_rescue_descend_rate)을 재사용해 자동으로 낮춘다.

11.2 동작 조건

조건	값
비행 모드	Alt Hold (ALT_HOLD_MODE)
Airmode	OFF (isAirmodeEnabled() == false)
시동	ARMED

세 조건을 모두 만족해야 "착륙 보조"로 간주한다. 하나라도 벗어나면 11.3절의 히스테리시스 래치가 즉시 모두 풀린다(altHoldInit()과 Alt Hold 종료 전환 시에도 풀림). 페일세이프 자동 착륙(failsafelsActive())과 GPS Rescue 자체의 속도 처리는 기존 로직 그대로이며 착륙 보조가 끼어들지 않는다. USE_GPS_RESCUE가 없는 빌드에서는 착륙 보조 속도 변경 코드 자체가 컴파일되지 않는다(#ifdef USE_GPS_RESCUE) — 이 저장소의 10개 기체는 전부 GPS/GPS Rescue를 포함하므로 전 기체 적용된다.

11.3 수직 속도 상한 및 히스테리시스

Alt Hold의 기존 수직 속도 상한($\text{alt_hold_climb_rate} \times 10 \text{ cm/s}$)을 고도($\text{getAltitudeCm}()$)에 따라 아래처럼 교체한다. 같은 상한이 상승에도 동일하게 적용되며 스틱 비율은 그대로 유지된다.

고도 구간	속도 상한	켜짐(ON)	꺼짐(OFF)
5m 초과 (또는 Airmode ON / Alt Hold 이상)	alt_hold_climb_rate 그대로	-	-
5m 이하	$\text{gps_rescue_descend_rate} \times 2$	$\leq 5.0 \text{ m (500cm)}$	$\leq 5.5 \text{ m (550cm)}$
2m 이하	$\text{gps_rescue_descend_rate} \times 1$	$\leq 1.8 \text{ m (180cm)}$	$\leq 2.2 \text{ m (220cm)}$

2m 구간이 켜져 있으면 5m 구간 안에서도 x1이 우선한다. 경계에서 상태가 떨어지 않도록 ON/OFF 임계값을 다르게 둔 히스테리시스 래치 두 개를 쓴다 — isAltHoldLandingMode() 호출마다 5m 래치가 갱신되고, 속도 상한 계산 함수가 이 함수를 호출한 뒤 2m 래치를 갱신하는 구조다.

11.4 OSD 경고 표시

isAltHoldLandingMode()가 true면 OSD 경고창에 "ALTHOLD : LANDING"(DISPLAYPORT_SEVERITY_INFO, 깜박임 없음)을 표시한다. 배터리·RSSI·페일세이프 경고보다 우선순위가 낮고, 비주얼 비퍼(* * * *) 바로 앞에 뜬다. Airmode가 켜지거나, 디스암되거나, 고도가 5.5m를 넘으면 문구가 사라진다. 새 OSD 경고 비트나 CLI 항목은 추가하지 않았다.

11.5 이식 시 구조 차이

2026.6.2는 이번 이식 기준점(v7, 커밋 415e8c4) 이후 flight_plan_nav/position_nav 모듈과 autopilotGetEffectiveHoverThrottlePwm() 등을 추가하며 getAltitudeCmControl(), altHoldState_t.maxClimbRate로 API가 크게 바뀌었다 — 2025.12.5에는 해당 모듈이 전혀 없다. 네 커밋을 직전 포팅 기준점과 비교해(git diff 415e8c4..90d5d9c, 이후 세 커밋은 git show) 착륙 보조 관련 변경만 분리해낸 뒤,

2025.12.5의 등가 API로 치환해서 이식했다: getAltitudeCmControl() -> getAltitudeCm()(2025.12.5의 유일한 고도 getter), altHold.maxClimbRate -> altHold.maxVelocity(이 저장소의 기존 필드명).
flightPlanNavGetRescueVerticalRateCmS() 기반의 GPS Rescue 대체 분기는 2025.12.5에 해당 기능 자체가 없어 제외했다 — 이 저장소의 GPS Rescue 자체 속도 처리는 변경하지 않으며, 착륙 보조는 Alt Hold 경로에만 적용된다.

```
// alt_hold_multirotor.c (신규)
#define ALT_HOLD_LANDING_ALT_HIGH_CM    500.0f // 5.0 m: on
#define ALT_HOLD_LANDING_ALT_OFF_CM     550.0f // 5.5 m: off
#define ALT_HOLD_LANDING_ALT_LOW_ON_CM  180.0f // 1.8 m: on
#define ALT_HOLD_LANDING_ALT_LOW_OFF_CM 220.0f // 2.2 m: off

static bool altHoldLandingLatched; // 5.0 m on / 5.5 m off
static bool altHoldLandingNearLatched; // 1.8 m on / 2.2 m off

bool isAltHoldLandingMode(void)
{
    if (!(ARMING_FLAG(ARMED) && FLIGHT_MODE(ALT_HOLD_MODE) && !isAirmodeEnabled())) {
        altHoldLandingLatched = false;
        altHoldLandingNearLatched = false;
        return false;
    }
    const float altitudeCm = getAltitudeCm();
    altHoldLandingLatched = altHoldLandingLatched
        ? (altitudeCm <= ALT_HOLD_LANDING_ALT_OFF_CM)
        : (altitudeCm <= ALT_HOLD_LANDING_ALT_HIGH_CM);
    return altHoldLandingLatched;
}

static float altHoldMaxClimbRate(void)
{
#ifdef USE_GPS_RESCUE
    if (isAltHoldLandingMode() && !failsafeIsActive()) {
        const float descendRateCmS = (float)gpsRescueConfig()->descendRate;
        const float altitudeCm = getAltitudeCm();
        altHoldLandingNearLatched = altHoldLandingNearLatched
            ? (altitudeCm <= ALT_HOLD_LANDING_ALT_LOW_OFF_CM)
            : (altitudeCm <= ALT_HOLD_LANDING_ALT_LOW_ON_CM);
        if (altHoldLandingNearLatched) {
            return descendRateCmS;
        }
        return descendRateCmS * 2.0f; // isAltHoldLandingMode()가 히스테리시스를 이미 적용
    }
#endif
    return altHold.maxVelocity;
}

// osd/osd_warnings.c (renderOsdWarning, 비주얼 비퍼 직전)
#ifdef USE_ALTITUDE_HOLD
    if (isAltHoldLandingMode()) {
        tfp_sprintf(warningText, "ALTHOLD : LANDING");
        *displayAttr = DISPLAYPORT_SEVERITY_INFO;
        return;
    }
#endif

```

11.6 시뮬레이션 테스트

2026.6.2 저장소 src/test/unit/althold_unittest.cc의 AltholdLandingAssist 스위트(10개)를 이 저장소의 AltholdCustomSim 픽스처를 상속하는 형태로 이식했다. gpsRescueConfig_t에 대한 PG_REGISTER와 isAirmodeEnabled()를 제어하는 testAirmodeEnabled 스텝을 추가하고, altHold.targetVelocity를 관찰하기 위해 altHoldState_t 앞부분 레이아웃과 일치하는 익명 구조체를 extern으로 선언했다. src/test/Makefile의 althold_unittest_DEFINES에 USE_GPS_RESCUE=를 추가했다. 호스트(PC, x86) 빌드라 실제 타깃 빌드와 무관하게 cd src/test && make test_althold_unittest 로 바로 실행할 수 있다.

검증한 10개 시나리오(전부 통과, 기존 13개 포함 총 23개): 5m 초과 시 alt_hold_climb_rate 그대로(700cm/s), 5m 이하 x2(150->300), 2m 이하 x1(150), Airmode ON이면 낮은 고도에서도 원래 값, 스틱 절반 입력 시 비율 유지(-75), 낮은 고도에서 스틱 풀업 시 상승 속도도 동일 상한 적용, isAltHoldLandingMode()가 Airmode/시동 상태를 따름, 5.0m 켜짐·5.5m 꺼짐 히스테리시스(경계 사이 상태 유지), 1.8m 켜짐·2.2m 꺼짐 히스테리시스, 히스테리시스가 속도 상한 자체에도 적용됨.

11.7 빌드/바이너리 검증

전 기체(10대) 보드별 slim 재빌드 성공(파일명 접미사 custom_v7_slim -> custom_v8_slim), 컴파일 경고/오류 없음, 플래시 오버플로 없음(24.37~78.97% 사용, v7 대비 기체당 +0.02~+0.38%p 수준). 4종 커스텀 CLI 파라미터와 신규 "ALTHOLD : LANDING" 문자열이 전 기체 hex 바이너리에 포함됨을 확인. #pragma message 진단 삽입(임시, 검증 후 원복)으로 isAltHoldLandingMode()가 전처리기 단계에서 실제로 컴파일됨을 확인.

11.8 권장 시험 절차 (9.7절에 추가)

① 프롭 제거 벤치에서 gps_rescue_descend_rate를 벤치 조건에 맞게 미리 설정한다. ② Airmode를 끈 채 Alt Hold에 진입하고, 블랙박스로 고도·altHold.targetVelocity·OSD "ALTHOLD : LANDING" 표시 구간을 함께 로깅하며 고도를 5m, 2m 경계로 천천히 오르내려 속도 배율 전환과 OSD 표시/소거 타이밍을 확인한다. ③ 5.0~5.5m, 1.8~2.2m 구간에서 고도를 작게 흔들어(진동) 배율이 떨어지지 않고 히스테리시스대로 유지되는지 확인한다. ④ Airmode를 켜 채로 같은 고도에서 재확인해 alt_hold_climb_rate 원래 값이 그대로 쓰이는지 확인한다. ⑤ 페일세이프를 유발해 페일세이프 자동 착륙 하강 속도가 착륙 보조의 영향을 받지 않는지 확인한다.

12. 부록: alt_hold_hover_throttle 검증과 적용 범위 — 2025.12.5 v9

이 장은 앞 장들의 alt_hold_hover_throttle 설명("0이 아니면 우선 적용" 등)보다 우선한다. 2026.6.2 커스텀 v15/v16의 수정을 2025.12.5 구조에 맞게 이식했다. CLI 항목 추가 없음, PG 버전 변경 없음.

12.1 허용값

항목	동작
허용값	0 또는 1100~1700 (경계 포함). CLI 범위 0~1700은 그대로
범위 밖 값(1, 500, 1099, 1701)	저장은 되지만 무시(0으로 간주) -> ap_hover_throttle 사용
0	ap_hover_throttle 상속(기존 동작)
스위치로 켜 Alt Hold / Position Hold	유효한 alt_hold_hover_throttle을 호버 기준으로 사용
페일세이프 착륙	failsafelsActive() 동안 ap_hover_throttle 사용, 해제되면 다시 alt_hold_hover_throttle
AUTOPILOT 모드	2025.12.5에는 AUTOPILOT_MODE가 없어 해당 없음
GPS Rescue	기존대로 ap_hover_throttle (변경 없음)

12.2 왜 바꿨나

CLI 범위가 0~1700이라 1이나 500 같은 값도 저장되고, 0이 아니면 검증 없이 호버로 쓰였다. 또 페일세이프 착륙도 ALT_HOLD_MODE를 켜므로 그때도 alt_hold_hover_throttle이 우선 적용되었다. 이 값은 조종자가 스위치로 켜 Alt Hold/Position Hold 의 스틱 감각을 맞추기 위한 값이므로, 자동 착륙에는 쓰지 않는다.

12.3 구현 (alt_hold_multirotor.c)

ALT_HOLD_HOVER_THROTTLE_VALID_MIN(1100)/VALID_MAX(1700), altHoldValidHoverThrottle(), altHoldGetHoverThrottle()을 추가했다. 2025.12.5에는 진입 순간 호버 캡처가 없고 altHoldInit()에서 한 번 고정하던 값을, 사용 시점마다 failsafelsActive()를 보고 정하도록 바꿨다. 해제 대기(10장)의 기준은 기존대로 ap_hover_throttle이다.

12.4 검증

호스트 유닛테스트 AltholdHoverThrottle 5개 추가(총 30개 통과): 유효값 1400 사용, 페일세이프 중 ap_hover_throttle 1300 및 해제 후 1400 복귀, 범위 밖 값 무시, 경계 1100/1700, 0 상속. 원래 코드에서는 4개가 실패함을 확인했다. 전 기체 custom_v9_slim 재빌드(오류/경고 0).

12.5 권장 시험

프롭 제거 벤치에서 alt_hold_hover_throttle을 1400으로 두고 스위치 Alt Hold 진입 시 블랙박스 debug(autopilot_altitude)의 호버 기준을 확인한다. 이어서 페일세이프(송신기 OFF)를 유발해 ap_hover_throttle 기준으로 바뀌는지, 복귀 시 다시 1400이 쓰이는지 확인한다.

13. 부록: 호버 변수 분리와 우선순위 정리 — 2025.12.5 v10

이 장은 12장(v9)과 앞 장들의 alt_hold_hover_throttle 설명보다 우선한다. 2026.6.2 커스텀 v15~v17의 수정을 2025.12.5 구조에 맞게 이식했다. CLI 항목 추가 없음, PG 버전 변경 없음.

13.1 변경 배경

v9는 alt_hold_hover_throttle과, ap_hover_throttle=0일 때의 진입 순간 스틱 위치(순정 동작)를 구별하지 못했고, GPS Rescue 플래그가 켜진 뒤 Alt Hold 작업이 해제 처리를 하기 전 한 주기 동안 전용 값이 읽힐 수 있었다.

13.2 호버 우선순위

순위	값	조건
1	altHoldOverrideHoverPwm (검증된 alt_hold_hover_throttle, 1100~1700)	조종자 스위치일 때만. failsafelsActive() 이거나 GPS_RESCUE_MODE 이면 건너뛴
2	ap_hover_throttle	0이 아니면
3	altHoldCapturedHoverPwm (진입 순간 스틱 값)	ap_hover_throttle=0일 때만 진입 시 캡처
4	기본값 1275	위 값이 모두 없을 때

두 변수는 Alt Hold 진입 시 채우고(전용 값이 있어도 스틱 캡처를 항상 수행), Alt Hold 종료와 초기화 시 모두 0으로 만든다. 호버값은 진입 순간에 확정되므로 Alt Hold 중 CLI로 값을 바꿔도 다음 진입부터 적용된다.

AUTOPILOT_MODE는 2025.12.5에 없어 해당 없음.

13.3 운용 요약

alt_hold_hover_throttle 허용값은 0 또는 1100~1700이고 범위 밖 값은 무시된다. 이 값은 조종자가 스위치로 켜 Alt Hold/Position Hold에서만 쓰이며, 페일세이프 착륙과 GPS Rescue는 ap_hover_throttle > 진입 순간 스틱 캡처값 > 기본값 순으로 쓴다.

13.4 검증

호스트 유닛테스트 AltholdHoverThrottle 9개(총 34개 통과): 유효값 사용, 페일세이프 중 ap_hover_throttle 사용 후 복귀, 범위 밖 값 무시, ap_hover_throttle=0일 때 진입 스틱 캡처(정상시 1400, 페일세이프 1200), 종료 시 초기화와 재캡처, GPS_RESCUE_MODE 플래그 즉시 제외. 전 기체 custom_v10_slim 재빌드.

14. 부록: 빌드 구성 · 제거 기능 · 배포 절차 (v4 포함)

이 장은 7장(기체별 매핑)의 상세 근거이며, 빌드 구성은 이 장이 우선한다. 모든 빌드는 보드별 `make <BOARD> EXTRA_FLAGS=...`로 만들며 스크립트는 `custom-patch/build_custom.sh`, 결과는 `custom-patch/firmware/2025.12.5/`에 생성된다.

14.1 v4: 전 기체 공통 제거 항목

옵션	방법	비고
USE_SERVOS	common_post.h의 CUSTOM_NO_SERVOS → #undef	서보 믹서 미사용 기체 정리. 전 기체 diff all 확인 완료
USE_BATTERY_CON TINUE	CUSTOM_NO_BATTERY_CONTINUE → #undef	브라운아웃 중 아밍 유지 기능, 미사용
USE_OSD_SD / USE_MAX7456	COMMON에 -DUSE_OSD_HD 선택의	common_pre.h의 'SD/HD 둘 다 미정이면 둘 다 켜다' 구조를 이용. SD가 안 켜지면 MAX7456도 자동 제거

디지털 VTX(MSP DisplayPort, Walksnail 등)만 쓰는 전 기체에 적용한 사용자 지시이며, 아날로그 OSD를 쓰는 기체에는 사용할 수 없다.

14.2 보드별 컴파일 옵션 요약

구분	옵션	대상
COMMON	-DCUSTOM_NO_* 계열(VTX 제어, 트랜스폰더, 레인지파인더 등), -DUSE_OSD_HD	전 기체
자력계	-DUSE_MAG (보드 config 빌드는 자동으로 켜지 않음)	Pavo25 V2 제외 9대
F722 추가	-DUSE_ALTITUDE_HOLD -DUSE_GPS -DUSE_POSITION_HOLD	F722 기체 6대(512KB라 공식 조건 TARGET_FLASH _SIZE>=1024 미충족)
PINIO 제거	CUSTOM_NO_PINIO	Mark4_6in, AOS_UL7_X8, Explorer LR4, X8_5INCH
LED 스트립 제거	CUSTOM_NO_LED_STRIP	MARIO5, AOS_UL7_O4, Mark4_6in, 8IN-KOPIS_X8, CHIMERA7, Pavo25 V2
수신기 프로토콜	기체별로 필요한 것만 포함	CRSF 7대(MARIO5 포함), FPort 2대(AOS_UL7_O4, X8_5INCH), SBUS 1대(Mark4_6in)

수신기 프로토콜은 사용자 확인 사항이다. MARIO5=CRSF, AOS_UL7_O4=FPort, X8_5INCH=FPort는 CLI diff에 `serialrx_provider`가 없어 가정한 값이므로, 틀리면 바인드가 안 된다. 플래시 직후 벤치에서 확인한다.

14.3 플래시 사용률 (v10)

F405(1MB) 39.2~39.7%, F722(512KB) 73.7~79.0%, H743(2MB) 24.4%. 기체별 수치는 7장 표 참고. 같은 MCU라도 보드·기체마다 hex가 다르므로 기체와 파일명 라벨(MARIO5/AOSUL7O4/PAVO25V2/EXPLORERLR4 등)을 반드시 대조한다. MARIO5와 AOS_UL7_O4는 같은 FC(SPEEDYBEEF405V4), Pavo25 V2와 Explorer LR4는 같은 FC(JHEF7DUAL)라서 보드명만으로는 구별되지 않는다.

14.4 배포 절차와 무결성

- 1) custom-patch/firmware/2025.12.5/ 에서 해당 기체의 *_v10_slim.hex 하나만 선택
 - 2) 같은 폴더의 SHA256SUMS.txt 로 무결성 확인: `sha256sum -c SHA256SUMS.txt`
 - 3) Configurator에서 Load Firmware [Local] 로 올림
 - 4) 플래시 후 CLI에서 보관해 둔 diff all 재적용 (오류 줄은 14.5 참고)
 - 5) 프롭 제거 상태로 벤치 확인: 수신기 바인드, OSD, 모터 방향, ALT/POS/ARM 스위치 배정
- 파일명 규칙: `betaflight_2025.12.5_<MCU>_<BOARD>_<VEHICLE>_custom_v10_slim.hex`. 버전 접미사는 v4부터 _vN_slim 형태로 한 단계씩 올라갔으며(v4~v10), 이전 버전의 hex는 저장소에서 제거되었다.

14.5 플래시 후 알려진 오류 줄 (무시 가능)

원인	오류가 나는 CLI 줄
VTX 제어 기능 제거	<code>osd_vtx_channel_pos</code> , <code>osd_sys_vtx_temp_pos</code> 등
LED 핀 없음(8IN-KOPIS_X8, Pavo25 V2)	<code>feature LED_STRIP</code> , <code>resource LED_STRIP 1 NONE</code>
PINIO 제거 기체(Mark4_6in, AOS_UL7_X8, Explorer LR4, X8_5INCH)	PINIO 관련 줄
v4 제거(전 기체)	<code>servo</code> , <code>smix</code> , <code>battery_continue</code> , 아날로그 OSD 관련 줄(있었다면)

위 줄 외의 오류(특히 `resource`, `serial`, `set alt_hold_*`, `set landing_*`, `set ap_*`)가 나오면 hex가 기체와 맞지 않을 수 있으므로 중단하고 파일명을 다시 확인한다.

15. 부록: 코드 검토 결과 · 착륙 보조 한계 · 컴파일 검증

v8 착륙 보조 이식 후 수행한 코드 검토에서 확인된 사항이다. 비행 안전에 영향이 있는 항목은 운용 시 유의한다.

15.1 착륙 보조의 속도 상한은 '목표고도 이동 속도' 제한이다

altitudeControl()에는 수직속도 제한이 없다. 따라서 v8의 속도 상한(5m 이하 $\times 2$, 2m 이하 $\times 1$)은 목표 고도가 움직이는 속도(slew)만 제한하며, 기체가 실제로 내려가는 속도를 직접 제한하지 않는다. 스틱을 급히 내리면 목표 고도는 천천히 내려가지만, 기체가 목표를 따라가는 PID 응답 때문에 순간 하강 속도는 이보다 커질 수 있다. 접지 직전의 충격 완화는 보조 효과로 이해하고, 최종 접지는 조종자가 스로틀로 관리한다.

15.2 고도 기준은 AGL이 아니라 시동(아밍) 지점

getAltitudeCm()은 아밍 지점 기준의 상대 고도다. 언덕·건물 위에서 아밍했거나 착륙 지점이 이륙 지점보다 높거나 낮으면 '5m/2m 이하' 판단이 실제 지면 높이와 어긋난다. 이륙 지점과 다른 높이에 착륙할 때는 착륙 보조의 구간 전환 시점이 실제와 다를 수 있다.

15.3 Airmode OFF가 저고도 상승도 함께 제한

착륙 보조는 'Alt Hold + Airmode OFF + 시동 중'이면 방향에 관계없이 같은 속도 상한을 적용한다. 따라서 저고도에서 스틱을 풀어도 상승 속도가 gps_rescue_descend_rate 기반 값으로 제한된다(고고도에서 airmode를 끄면 이 구간에서만 영향). 위급하게 빠른 상승이 필요하다면 Alt Hold를 해제하거나 airmode를 켜고.

15.4 isAltHoldLandingMode()의 부수 효과

위치	영향
alt_hold_multirotor.c	호출될 때마다 5.0/5.5m 래치를 갱신(상태 있는 함수). 호출 빈도와 무관하게 결과는 동일하나, 모드 종료·초기화 분기에서 래치를 반드시 해제해야 함(구현 완료)
osd_warnings.c	경고 문자열 'ALTHOLD : LANDING' 표시를 위해 같은 함수를 호출 → OSD 갱신 주기에도 래치가 갱신됨. 우선순위는 배터리/RSSI/페일세이프 경고보다 낮음
failsafelsActive()	페일세이프 중에는 착륙 보조의 속도 교체 안 함(페일세이프/GPS Rescue 하강 로직이 우선)

15.5 처리 주기 표기 정정

해제 대기 판정(altHoldRequestActive)은 processRxModes()(RX 태스크)에서 호출되므로 100 Hz 고정이지 않고 수신 프레임 속도에 연동한다. v7 설명의 '100Hz, 10ms' 표기는 대략적 설명이며, 스틱 가로지름 판정은 직전 RX 샘플과 비교하므로 RX 속도가 달라도 동일하게 동작한다. Alt Hold 목표고도 갱신(altHoldUpdateTargetAltitude)은 PID/Alt Hold 작업 주기를 따른다.

15.6 컴파일 검증 방법 (보드별 매크로)

보드·기체마다 옵션이 다르므로 'make -n -B' 명령에서 컴파일 명령을 가져와 '-E -dM'으로 바꿔 보드별 최종 매크로를 덤핑하여 확인했다. 또한 hex를 Intel-HEX로 해석하는 스크립트로 hex 안의 CLI/OSD 문자열을 검색했다.

확인 항목	방법	결과
USE_SERVOS/BATTERY_CONTINUE/ OSD_SD/MAX7456 제거	보드별 -E -dM 매크로	전 기체 미정의, USE_OSD_HD만 정의
USE_ALTITUDE_HOLD / POSITION_HOLD / GPS	보드별 매크로	전 기체 정의(F722는 추가 플래그로 컴)
USE_MAG	보드별 매크로 + 심볼	Pavo25 V2만 미정의

확인 항목	방법	결과
CLI/OSD 문자열 5종(alt_hold_deadband_low, alt_hold_full_low_is_max_descend, alt_hold_hover_throttle, landing_disarm_airmode_off_only, ALTHOLD : LANDING)	hex 문자열 검색	전 기체 포함
core.c BOXMAG → MAG_MODE 블록	소스 확인(v9)	정상 존재
호스트 유닛테스트	src/test/unit/althold_unittest.cc	34개 전부 통과(AltholdHoverThrottle 9개 포함)

유닛테스트는 로직 검증이며 IMU/RC 잡음, 스케줄러 타이밍은 반영하지 않는다. 실비행 확인 범위는 16.3 참고.

참고: v9에서 CHIMERA7 플래시 사용률이 v8보다 0.29%p 낮아졌다. 기능 문자열은 동일해 원인은 LTO 배치 차이로 추정되나 확정하지 못했다.

16. 부록: 운용 메모 (랜딩 디스암, min_check, 실비행 확인) 및 버전 이력

16.1 랜딩 디스암(EZ Disarm) 동작 조건

pid.c의 호출 조건은 다음과 같다.

```
if (pidRuntime.useEzDisarm &&
    (!pidRuntime.landingDisarmAirmodeOffOnly || !isAirmodeEnabled()))
    disarmOnImpact();
```

질문	답
airmode_off_only는 airmode OFF일 때만 작동?	설정이 ON일 때만 그렇다. 기본 OFF에서는 airmode 상태와 무관하게 작동. ON이고 airmode가 켜져 있으면 디스암 판정이 호출되지 않는다
threshold 값이 있어도 airmode ON이면 안 되나?	airmode_off_only = ON일 때 맞다. OFF이면 작동한다. 프로파일별 값이므로 사용하는 프로파일 모두 확인
스로틀 값에 영향을 받나?	받는다. disarmOnImpact()는 ① wasThrottleRaised() ② (스틱 중앙 5% 미만 AND 스로틀 5% 미만) 또는 Alt Hold 모드 ③ 저크 × 민감도(Alt Hold 중 착지 고도 이하 1.5) > threshold × 10, 세 조건이 모두 만족될 때 디스암. 스로틀을 올려 둔 일반 모드에서는 디스암되지 않으며, Alt Hold 중에는 호버 스로틀 때문에 스로틀 조건이 면제된다
landing_disarm_threshold = 0	EZ Disarm 자체 비활성(공식 기본값). 권장 기본 CLI는 45 + airmode_off_only ON(5.1절)

16.2 min_check / max_check 공식 기본값

Betaflight 2025.12.5 공식 기본값은 min_check 1050, max_check 1900이다(pg/rx.c). 본 빌드는 이 값을 바꾸지 않으며, 권장 CLI(5장)도 min_check 1050이다. min_check는 THROTTLE_LOW 경계이므로 alt_hold_full_low_is_max_descend가 처리하는 구간의 하단 경계가 된다.

16.3 실비행 확인 결과와 미검증 항목

2026-10-02 사용자 보고: 실제 비행까지 모두 테스트했고 정상 동작. (v10 기준, 상세 항목은 사용자가 확인한 범위)

아래는 문서상 아직 별도 확인이 없는 항목이다. 해당 기능을 쓰기 전에 벤치 또는 단계적 비행으로 확인한다.

항목	권장 확인
Pavo25 V2 (자력계 없음) Position Hold 헤딩 추정	벤치 후 저고도 단계 시험. pos_hold_without_mag = OFF 설정과 동작 관계 확인
MARIO5/AOS_UL7_O4/X8_5INCH 수신기 프로토콜 가정	플래시 직후 바인드 확인
Pavo25 V2 CLI가 4.5.5 기준(오래됨)	최신 diff all로 재대조 후 적용
EZ Disarm(threshold 45) 실제 착지 판정	블랙박스로 접지 충격 크기 확인 후 임계값 조정(airmode_off_only ON 비행)
페일세이프 중 호버 기준(v9/v10)	실제 페일세이프 시험은 위험하므로 벤치/시뮬레이션 위주, 호스트 테스트는 통과

16.4 권장 기본 CLI 확정 (2026-10-02)

사용자 확정 값은 5.1절에 있다. 핵심: landing_disarm_threshold 45 + landing_disarm_airmode_off_only ON, alt_hold_deadband_low 0, alt_hold_full_low_is_max_descend ON, alt_hold_hover_throttle 1400, mixer_type EZLANDING(10/30). README 3-1절도 같은 값으로 맞췄다. 프로젝트 문서 '기본_추가CLI.txt'(09-30)는 landing_disarm_threshold 60이므로 45로 갱신해야 한다.

16.5 버전 이력 (2025.12.5 커스텀)

버전	내용	커밋
v1~v3	CLI 파라미터 4종(1, 2, 3장)	—
v4	서보/배터리-컨티뉴/아날로그 OSD 제거, 보드별 slim 빌드	—
v5	진입 스틱 래치	—
v6/v7	해제 대기 + OSD ALT WAIT / 호버 구간 건너뛸 수정	—
v8	착륙 보조 + OSD ALTHOLD : LANDING	—
v9	alt_hold_hover_throttle 검증(0 또는 1100~1700), 스위치 전용	3950e1272
v10	호버 변수 분리, 우선순위(override > ap_hover > captured > 1275)	4ac971a20